

MP1 Report: Small Language Model Challenge

****Course:**** DASE7506 Advanced Machine Learning

****Student ID:**** 3036843288

****Date:**** September 2026

1. Summary

Starting from the provided 4-layer GPT-2 baseline (test BPB ≈ 2.10), we reduce test BPB to ****1.52468**** on WikiText-2 (BPE-2048, independent 256-token causal windows, FP32 CPU evaluation). The final model is a two-component system:

1. A 10-layer SwiGLU transformer with RoPE, RMSNorm, and weight tying, trained from scratch on the supplied training text (4.82 M neural parameters).
2. A train-derived Kneser - Ney n-gram model (orders 2 - 10) combined with a recency cache and a copy mechanism, interpolated with the neural distribution at inference time.

All statistical tables are fit ****only on the supplied training split****; all hyperparameter and architecture choices are selected on the ****validation**** split; the test split is evaluated exactly once on the frozen bundle.

Model	val BPB	test BPB
--- ---	---:	---:
Baseline (provided)	2.071	~ 2.10
Final submission	**1.507**	**1.52468**

2. Method

2.1 Neural backbone

The neural component is a decoder-only transformer, identical in shape to a small LLaMA block:

- ****Vocabulary / context:**** fixed by the protocol at 2048 BPE types and 256 tokens.
- ****Width 192, 6 heads, head_dim 32, depth 10**** (final).
- ****RoPE**** (GPT-NeoX half-split convention) on queries and keys; no learned position embedding.
- ****RMSNorm**** instead of LayerNorm (no mean centering, no bias).
- ****SwiGLU**** MLP: $\text{down}(\text{silu}(\text{gate}(x)) * \text{up}(x))$ with hidden dim $\approx 8/3 \cdot \text{width}$.
- ****No bias**** on QKV/projection/MLP linears.
- ****Weight tying****: the output projection weight shares the token embedding.
- ****GPT-2 scaled residual init****: residual projections (attention output, MLP down-proj) initialized with $\text{std} = 0.02 / \sqrt{2 \cdot \text{depth}}$.

- **Dropout 0.1** on both residual branches.

2.2 Statistical mixture

The neural distribution is interpolated with train-derived statistical distributions, all built from the training split only:

- **Kneser - Ney smoothed n-grams**, orders 2 through 10. Orders 6 - 10 use open-addressing hashing with exact-token collision rejection, so a hash collision falls back to back-off rather than borrowing another phrase's counts.
- **Recency cache**: a small exponential cache over the current evaluation window (half-life 0, i.e. immediate cache recall), weighted by `cache_weight = 0.05`.
- **Copy mechanism**: re-weights the neural distribution toward tokens seen earlier in the same window, weighted by `copy_weight = 0.4`, gated by `copy_agreement = 0.5`.
- **Gated interpolation**: the n-gram component is gated by `ngram_gate = 0.5`; `ngram_weight = 0.2`. These three gating weights were selected by a $3 \times 2 \times 2$ grid ($\text{ngram_gate} \in \{0, 0.5, 1.0\}$, $\text{copy_half_life} \in \{0, 64\}$, $\text{copy_agreement} \in \{0, 0.5\}$) on the validation split only.

At inference the two components are blended per-token: the neural log-probabilities are converted to probabilities, multiplied and boosted by the n-gram/cache/copy terms, then renormalized. No cross-window state is carried; each 256-token window starts from a clean cache.

2.3 Training

- **Optimizer**: AdamW (lr = $1e-3$ for neural-only runs, $5e-5$ for the final continuation; weight decay 0.1, grad clip 1.0).
- **Schedule**: linear warmup (100 steps, or 50 steps for the fine-tuning phase) followed by cosine decay to 10% of peak.
- **EMA**: exponential moving average of neural weights with decay 0.99; the EMA weights are selected at the end.
- The neural backbone was trained in stages: first from scratch to 4000 steps at width 128/depth 6, then expanded to width 192/depth 8 and trained to 4000 steps, then expanded to depth 10 and fine-tuned for 1000 steps at low LR. The final checkpoint records 54,067,200 cumulative neural training targets (45,875,200 inherited plus 8,192,000 in the depth-10 stage) and 22,095 seconds (6.14 hours) cumulative neural training time. The paired depth experiment trained both branches on equal targets; only the selected depth-10 branch contributes to final checkpoint ancestry.

3. Ablation

All numbers are validation BPB on the same WikiText-2 validation split (376,599 targets, 1,148,007 bytes). Baseline uses the provided `model.py` unchanged. Every row below uses seed 17 unless noted.

#	Model	Params	Train tokens	val BPB	Δ vs previous
0	Baseline GPT (learned pos emb, LayerNorm, GELU, depth 4)	1.09 M	9.8 M	2.071	
1	+ RoPE (depth 4, 1200 steps)	1.06 M	9.8 M	1.923	-0.148
2	+ RoPE, longer (2400 steps)	1.06 M	19.7 M	1.780	-0.143
3	+ RMSNorm + SwiGLU + scaled init, depth 6, 2400 steps	1.54 M	19.7 M	1.706	-0.074
4	Same as #3, 4000 steps	1.54 M	32.8 M	1.677	-0.029
5	Wider/deeper: width 192, heads 6, depth 8, dropout 0.1, 4000 steps	3.94 M	32.8 M	1.612	-0.065
6	+ Kneser-Ney n-gram (orders 2-10) + cache + copy	3.94 M + 41 MB tables	45.9 M	1.508	** -0.104 **
7	+ grid search of gating weights (12 candidates)	same	same	1.508	-0.000
8	+ expand neural to depth 10, fine-tune 1000 steps, EMA selected	4.82 M + 41 MB tables	54.1 M cumulative	**1.507**	-0.001

Test BPB of the frozen bundle (#8): 1.52468 (validation 1.507; generalization gap 0.018).

3.1 Paired comparison: depth 8 vs depth 10

The final neural fine-tuning was run as a paired experiment: both branches started from the same width-192 checkpoint, used identical seed 31, identical training batches (verified by chained sampling digest), and were evaluated at the same validation checkpoints (steps 400, 800, 1000):

Step	depth 8 (EMA)	depth 10 (EMA)
400	1.50778	1.50767
800	1.50723	1.50709
1000	1.50704	**1.50688**

Depth 10 wins by 0.00016 at the final checkpoint. The difference is within run-to-run noise and confirms the neural component has saturated at this capacity.

3.2 Validation curves

- Row #4 (depth 6, width 128): 1.848 $\rightarrow$ 1.751 $\rightarrow$ 1.711 $\rightarrow$ 1.685 $\rightarrow$ 1.677 at steps 800/1600/2400/3200/4000.
- Row #5 (depth 8, width 192): 1.902 $\rightarrow$ 1.776 $\rightarrow$ 1.725 $\rightarrow$ 1.706 at steps 600/1200/1800/2400.

Training loss (neural-only, row #4) reached 3.03 nats/token while validation was 3.54, indicating mild overfitting; this motivated dropout 0.1 in the wider model.

4. Resource measurements

Measured on the submission machine (4-thread CPU, torch 2.7.1+cpu, FP32):

Metric	Value	Limit
Test scoring time	40.108 s	$\leq 5\times$ baseline (≈ 78 s on this machine)
Peak evaluation RAM (depth-10 preflight)	1.80 GB	≤ 4 GiB
Inference assets (checkpoint + code)	60,310,478 bytes (57.52 MiB)	≤ 64 MiB
Parameters	4.82 M neural + sparse n-gram tables	—

The n-gram tables consume ~ 41 MB of the inference bundle and the neural weights make up most of the remainder. The system is **asset-bound**, not **compute-bound**: the bundle is close to the 64 MiB limit, leaving limited room to grow either the neural model or the n-gram tables.

5. Critical analysis

5.1 What helped, and by how much

The single largest improvement came from **adding the statistical n-gram/copy mixture** (-0.104 BPB, row 5 $\rightarrow$ 6). This is consistent with the literature: on a small corpus like WikiText-2, a well-smoothed Kneser - Ney model already captures most local phrase structure, and the neural model contributes mainly on top of it. The copy mechanism helps reproduce rare tokens that recur within a 256-token window.

Among neural changes, **RoPE** was the most valuable single modification (-0.148), followed by **wider/deeper architecture with SwiGLU/RMSNorm** (-0.065 to -0.104). Longer training had diminishing returns (row 3 $\rightarrow$ 4: -0.029).

5.2 What did not help

- Further neural training after the n-gram mixture was fitted (row 7 $\rightarrow$ 8: -0.001). The neural model had already converged; the remaining error is dominated by tokens that neither the local n-gram nor the small network can predict.
- Grid search over mixture weights after the initial fit changed BPB by less than 0.002.
- Depth 8 vs 10 was a wash (0.0002 difference).

The final depth-10 checkpoint reaches validation BPB 1.506880 and test BPB 1.52468. The adaptive-mixture selection used 12 declared validation configurations; its selection process took 70.7 seconds, with a separate original-scorer/resource verification process of 392.4 seconds. Training and search run details, including

earlier stages and the paired depth experiment, are recorded in the README and runs audit files.

5.3 Trade-offs

- **Asset size vs quality:** the n-gram tables are 41 MB. Shrinking them (fewer orders, more aggressive hashing) would free space for a larger neural model but would lose more BPB than it gains.
- **Scoring time:** the hybrid mixture adds $\sim 2\times$ over a pure neural forward pass (n-gram lookup is cheap but the cache/copy walks the window). We use $2.6\times$ of baseline budget.
- **No external data / no retrieval:** the 256-token independent-window rule prevents any long-context or retrieval trick. All gains come from architecture and train-derived statistics.

5.4 Why we did not reach lower

A ~ 1.5 BPB result is near the practical floor for this setup. Reaching 1.3 would require either (a) a much larger neural model, which the 64 MB asset ceiling forbids once n-gram tables are included, or (b) test-time tricks that the fixed evaluator disallows. We consider 1.52468 a strong result for a 4.8 M parameter CPU-only model.

6. Reproduction instructions

```
```bash
cd code
python -m venv .venv
.venv\Scripts\Activate.ps1 # Windows PowerShell
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt

Verify correctness (5 contract tests, no data download)
python -m unittest discover -s tests -v

Reproduce the reported test score (no retraining needed)
python evaluate.py --checkpoint runs/final-submission/checkpoint.pt \
 --split test --device cpu --precision fp32 --threads 3
```
```

The bundle `runs/final-submission/` contains:

- `checkpoint.pt` — frozen EMA weights (neural + n-gram tables + cache), 60.3 MB.
- `hybrid.py` — the mixture model wrapper.
- `student.py` — the neural backbone.
- `README.txt` — one-line command.

The training recipe and all intermediate checkpoints are in `runs/`; the exact commands

for each stage are recorded in the corresponding `metrics.json` under the `command` key.

7. AI assistance disclosure

This project was developed interactively with an AI coding assistant. The assistant helped with: explaining the baseline data pipeline and BPB metric, implementing RoPE/RMSNorm/SwiGLU from scratch, writing the Kneser - Ney mixture and cache/copy code, and running paired ablations. The author verified every change against the provided contract tests, selected all hyperparameters on the validation split, and understands the implementation.